Document number: P0046R0
Date: 2015-09-10
Project: Programming Language C++, Library Evolution Working Group
Reply-to: *Tomasz Kamiński <tomaszkam at gmail dot com>*

Change is_transparent to metafunction

Introduction

This proposal discusses an alternative design for enabling a heterogeneous lookup function in the associative containers, that relies on use of metafunction.

This paper is result of the LWG recommendation provided in the resolution of the LWG issue #2430.

Motivation and Scope

In the scope of this paper we propose a new metafunction `permits_heterogenous_lookup`. This metafunction is designed to be used to enable the heterogeneous lookup in associative container, instead of the current standard method relying on presence of `is_transparent` nested type in the comparator type ($23.2.4 [associative.reqmts]).

Preserving backward compatibility

The addition of the heterogeneous lookup to the container may silently change the behavior of the existing code and, to avoid such situations, it was designed as an opt-in feature. However, the mechanism created for the purpose is not always working correctly - existing programs may define `is_transparent` nested type in their comparator types, and as a consequence the meaning of such programs will be changed.

The introduction of the type trait allows the users to resolve the backward compatibility problem described above via providing an explicit specialization of the trait that derives from `std::false_type`. It is worth noting that this is only a viable solution for the situation when the programmer does not have the control over the comparator's code.

Furthermore, it is possible to avoid false positives via providing the default definition of `permits_heterogeneous_lookup` that has BaseCharacteristic of `std::false_type`. However, such design will no longer support user-defined comparators that intentionally enable new lookup function by declaring nested type `is_transparent`.

The resolution of this problem is discussed in Backward compatibility subsection of the proposal.

Support for third party comparators

In the current design enabling heterogeneous lookup requires a modification of the comparator class to declare a nested type. As a consequence of this intrusiveness, discussed functionality cannot be enabled for the third-party types that are outside of their control, even if they support such operations.

In contrast, the use of a type trait allows the new lookup mechanism to be enabled in the program without any modification to the third party libraries.

Creating transparent call wrappers

The introduction of the type trait would also simplify the definition of call wrapper types that would like to 'inherit' the heterogeneous comparison trait from the wrapped callable.

Suppose we are creating `call_wrapper<F>`. In the case of the proposed `permits_heterogeneous_lookup` trait, the implementation is pretty straight-forward:

```
namespace std
{
  template<typename F>
  struct permits_heterogeneous_lookup<call_wrapper<F>>
    : permits_heterogeneous_lookup<F> {};
}
```

But for current design it becomes more complicated. The user is required to declare `is_transparent` nested type in the definition of `call_wrapper<F>` if such type was defined in the `F` type.

```
namespace detail
{
  template<typename F, typename IT = void>
  struct is_transparent_base {};


  template<typename F>
  struct is_transparent_base<F, void_t<typename F::is_transparent>>
  {
    using is_transparent = typename F::is_transparent;
  };
}

template<typename F>
struct call_wrapper : detail::is_transparent_base<F>
{ /*...*/ };
```

In addition to being longer and intrusive, the nested type implementation requires of type `F` to be complete at the point of instantiation of `call_wrapper<F>`. As a consequence, it is impossible to create a reference wrapper class that would support both incomplete types and heterogeneous container lookup.

Consistency with the Standard

Proposed solution will make this functionality consistent with rest of the standard that relies on the usage of the traits like: `is_placeholder` and `is_bind_expression` in similar situations.

Design Decisions

Semantics of the new trait

Existing `is_transparent` nested was used to mark classes that has one of the following characteristics:

1. Transparently wraps operation expressed using build-in operators.
2. Represents weak total order relation on heterogeneous types and as consequence support lookup thought a different type.

Although we may point out types that fulfills only one of above criteria. For example diamond version `logical_not` or `plus` are transparently wrapping some build-in operators, but usually they cannot be used as comparators. From the other side we may imagine the `icase_less` comparator that would provides overloads for `std::string` and `char*`.

The trait proposed in this paper is designed to identify only the types in the second category. The relation with the existing `is_transparent` is preserved only for backward compatibility reasons.

Naming

Following criteria was considered in the process of name selection for new metafunction:

- The name should not be too generic, to not collide with existing trait or reserve name that would be more suitable in other context. Names that was rejected because of this criteria: `is_generic`, `is_polymorphic`, `is_transparent`.
- The name should match the semantics of the metafunction: if its returns true, the container specialized with given functor will expose heterogeneous lookup function, not the functor object itself. Names that was rejected because of this criteria: `supports_heterogeneous_lookup`, `provides_heterogeneous_lookup`, `allows_heterogeneous_lookup` and `enables_heterogeneous_lookup`.
- The name should allow usage of metafunction for future extension that may provide heterogeneous lookup for other container types like `unordered_set`, as consequence it should not refer to concept specific to one implementation. Names that was rejected because of this criteria: `is_heterogeneous_comparator`.
- The metafunction represents an opt-in functionality, so it is not automatically enabled for every functor thats support heterogeneous comparison. As consequence set of types for which this functionality is enabled is subset of types that could be potentially used in this context. Names that was rejected because of this criteria: `is_compatible_with_heterogeneous_lookup`.

This paper propose a new name for the trait `permits_heterogeneous_lookup`, instead of keeping the name used for nested type. This resolution was selected because the original name (`is_transparent`) does not fulfill criteria of a good name listed above.

Backward compatibility

As the result of the introduction of `is_transparent` tag in the C++14 the design of `permits_heterogeneous_lookup` must carefully decide to choose one of the following:

1. Fix the pre-C++14 code compatibility problem that may be the result of silently enabling heterogeneous container lookup for existing comparators that have coincidentally provided the declaration of `is_transparent` nested type. To achieve that, the default implementation of the metafunction should always return `false`.
2. Preserve the support for post-C++14 user-defined comparator types that intentionally enable heterogeneous container lookup via the declaration of `is_transparent` nested type. To achieve that, the default implementation of the metafunction should return `true` for the functors with this type present.

Considering small possibility of occurrence of compatibility problems and the fact the harm was already done (C++14 was shipped), the proposal decides to choose second option and provide the default implementation that relies on presence of the nested type.

As a consequence of this decision, two ways of enabling heterogeneous lookup are provided by the standard: via the declaration of the nested type, and the explicit specialization of the trait. The author strongly believes that new mechanism provides an better alternative and decides to express this view via non normative note in the wording.

Place of declaration

The semantics of new `permits_heterogeneous_lookup` trait is directly related to the associative containers and it would be preferable to declare it in related header. However currently no common utility header for associative container is defined in the standard. Furthermore the scale of the proposed changes does not not justify introduction of the new header. As consequence we propose that `permits_heterogeneous_lookup` should be included in `<functional>` header, as it contains definitions of default hash function and comparators that currently enables heterogeneous lookup.

Impact On The Standard

This proposal has no dependencies beyond a C++11 compiler and Standard Library implementation.

Nothing depends on this proposal.

Proposed wording

The proposed wording changes refer to N4527 (C++ Working Draft, 2015-05-22).

After the declaration of `hash<T*>` in the section 20.9 [function.objects]/2 (Header `<functional>` synopsis), add:

```
  // 20.9.14, heterogeneous container lookup;
  template <class F> struct permits_heterogeneous_lookup;
```

After section 20.9.13 Class template `hash`, insert a new section.

20.9.14 Class template `permits_heterogeneous_lookup` [containers.heterlookup]

```
namespace std {
  template <class F> struct permits_heterogeneous_lookup; // see below
}
```

The associative containers defined in [associative] 23.4 use `permits_heterogeneous_lookup` to detect if member functions exposing heterogeneous container lookup should be enabled for a given comparator type.

Instantiations of the `permits_heterogeneous_lookup` template shall meet the `UnaryTypeTrait` requirements ([meta.rqmts] 20.10.1). The implementation shall provide a definition of `permits_heterogeneous_lookup<F>` that has a `BaseCharacteristic` of `true_type` if the *qualified-id* `F::is_transparent` is valid and denotes a type ([temp.deduct] 14.8.2), otherwise it shall have a `BaseCharacteristic` of `false_type`. A program may specialize this template for a user-defined type to have a `BaseCharacteristic` of `true_type` or `false_type`.

[*Note:* Specializing `permits_heterogeneous_lookup` usually provides a better solution than declaring `is_transparent` nested type. — *end note*]

Change the paragraph 23.2.4 Associative containers [associative.reqmts] p8.

In Table 102, `X` denotes an associative container class, `a` denotes a value of `X`, `a_uniq` denotes a value of `X` when `X`

supports unique keys, `a_eq` denotes a value of `X` when `X` supports multiple keys, `a_tran` denotes a value of `X` when the ~~*qualified-id* `X::key_compare::is_transparent` is valid and denotes a type (14.8.2)~~ `permits_heterogeneous_lookup<typename X::key_compare>::value` is `true` ([containers.heterlookup] 20.9.14), `i` and `j` satisfy input iterator requirements and refer to elements implicitly convertible to `value_type`, `[i,j)` denotes a valid range, `p` denotes a valid const iterator to `a`, `q` denotes a valid dereferenceable const iterator to `a`, `[q1, q2)` denotes a valid range of const iterators in `a`, `il` designates an object of type `initializer_list<value_type>`, `t` denotes a value of `X::value_type`, `k` denotes a value of `X::key_type` and `c` denotes a value of type `X::key_compare`; `kl` is a value such that `a` is partitioned (25.4) with respect to `c(r, kl)`, with `r` the key value of `e` and `e` in `a`; `ku` is a value such that a is partitioned with respect to `!c(ku, r)`; `ke` is a value such that `a` is partitioned with respect to `c(r, ke)` and `!c(ke, r)`, with `c(r, ke)` implying `!c(ke, r)`. `A` denotes the storage allocator used by `X`, if any, or `std::allocator<X::value_type>` otherwise, and `m` denotes an allocator of a type convertible to `A`.

Auxiliary proposal — `permits_heterogeneous_lookup_v` variable template

We propose `permits_heterogeneous_lookup_v` variable template as an auxiliary proposal. This is to give the Committee the freedom to make the decision to accept or not new metafunction independently of the inclusion into standard of C++ Extensions for Library Fundamentals, that defines variable templates for other standard traits.

After the declaration of `permits_heterogeneous_lookup<T>` in the section 20.9 [function.objects]/2 (Header `<functional>` synopsis), add:

```
  template <class F> constexpr bool permits_heterogeneous_lookup_v
    = permits_heterogeneous_lookup<F>::value;
```

Feature-testing recommendation

For the purposes of SG10, we recommend the macro name `__cpp_lib_permits_heterogeneous_lookup` to be defined in the `<functional>` header.

Usage example:

```
namespace third_party
{
  struct icase_less
  {
    bool operator()(std::string const&, std::string const&) const;
    bool operator()(std::string const&, char const*) const;
    bool operator()(char const*, std::string const&) const;
    bool operator()(char const*, char const*) const;
  };
}

#if __cpp_lib_permits_heterogeneous_lookup
namespace std
{
  template<>
  struct permits_heterogeneous_lookup<third_party::icase_less>
    : std::true_type {};
}
#else
struct my_icase_less : third_party::icase_less
{
  using is_transparent = void;
};
#endif
```

Acknowledgements

Jonathan Wakely provided an improved wording for the LWG issue #2430, from which this paper originates. Futhermore he has offered many useful suggestions and corrections to the proposal.

Andrzej Krzemieński offered many useful suggestions and corrections to the proposal.

Stephan T. Lavavej suggested numerous corrections to the proposed wording and addition of `permits_heterogeneous_lookup_v` variable template.

References

1. Jeffrey Yasskin, "Programming Languages — C++ Extensions for Library Fundamentals DTS" (N4480, http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4480.html)
2. Marshall Clow, "C++ Standard Library Active Issues List (Revision R92)" (N4383, http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4383.html)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4527, http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4527.pdf)